DevOps Engineering

DevOps Engineering: Reference-1

	Engineering DevOps: From Chaos to Continuous Improvement and Beyond
	Part-01: What is Engineering DevOps, and Why is It Important?
	What is Engineering DevOps?
01	Introduction
02	DevOps Engineering Blueprint
03	DevOps Engineering Tenets and CALMS
04	Origins of DevOps form an Engineering Point of View
05	The Dilemma of Defining Engineering DevOps
06	DevOps Engineering Terms
	Nine Pillars of Engineering DevOps
07	Introduction
08	Leadership Pillar
09	Collaborative Culture Pillar
10	Design for DevOps Pillar
11	Continuous Integration (CI) Pillar
12	Continuous Testing (CT) Pillar
13	Elastic Infrastructure (EI) Pillar
14	Continuous Monitoring (CM) Pillar
15	Continuous Security Pillar
16	Continuous Delivery (CD) Pillar
	Why is Engineering DevOps Important?>
17	Introduction
18	Engineering DevOps Myths and Realities
19	How Will I Know When I Have Engineered DevOps?
20	Benefits of Well-Engineered DevOps – ½
21	Benefits of Well-Engineered DevOps – ½
22	Cost of Not Engineering DevOps Properly
	Part-02: Engineering People, Process, and Technology for DevOps
	How Should People, Process, and Technology be Engineered for DevOps?
23	Does DevOps Engineering Require People to be Engineers?
24	DevOps People, Process, and Technology Engineering Maturity Levels
25	Three Dimensions of Engineering DevOps – People, Process, and Technology – People
26	Three Dimensions of Engineering DevOps – People, Process, and Technology – Process
27	Three Dimensions of Engineering DevOps – People, Process, and Technology – Technology
28	Twenty-Seven DevOps Engineering Critical Success Factors
29	Learn DevOps Value-Stream Pipeline Engineering – 1/3
30	Learn DevOps Value-Stream Pipeline Engineering – 1/ 3
31	Learn DevOps Value-Stream Pipeline Engineering – 1/3
	Value-Stream Management (VSM)
32	Why Is Value-Stream Management Important to DevOps?
33	How Does Value-Stream Management Work with DevOps? – ½
34	How Does Value-Stream Management Work with DevOps? – ½
35	What is Needed to Engineer a Value-Stream Management Solution for DevOps?
	Application Release Automation (ARA)
36	Why is Application Release Automation Important?
37	How Does Application Release Automation Work?

DevOps Engineering: Reference-2

DevOps Engineering: Reference-3

Software Testing

Software Testing: Reference-1

	Software Testing: Principles and Practices
	Testing Methodology
	Introduction to Software Testing
01	Introduction
02	Evolution of Software Testing
03	Software Testing – Myths and Facts
04	Goals of Software Testing
05	Psychology for Software Testing
06	Software Testing Definitions
07	Model for Software Testing
08	Effective Software Testing vs. Exhaustive Software Testing
09	Effective Testing is Hard
10	Software Testing as a Process
11	Schools of Software Testing
12	Software Failure Case Studies
	Software Testing Terminology and Methodology
13	Software Testing Terminology
14	Software Testing Life Cycle (STLC)
15	Software Testing Methodology
	Verification and Validation
16	Verification and Validation Activities
17	Verification
18	Verification of Requirements
19	Verification of High-Level Design
20	Verification of Low-Level Design
21	How to Verify Code
22	Validation
	Testing Techniques
	Dynamic Testing: Black-Box Testing Techniques
23	Boundary Value Analysis (BVA)
24	Equivalence Class Testing
25	State table-Based Testing
26	Decision Table-Based Testing
27	Cause-Effect Graphing Based Testing
28	Error Guessing
	Dynamic Testing: White-Box Testing Techniques
29	Need of White-Box Testing
30	Logic Coverage Criteria
31	Basis Path Testing
32	Graph Matrices
33	Loop Testing
34	Data Flow Testing
35	Mutation Testing
	Static Testing
36	Inspections
37	Structured Walkthroughs
38	Technical Reviews

Software Testing: Reference-2

Software Project Management

Software Project Management: Reference-1

	Applied Software Project Management
	Introduction
01	Tell Everyone the Truth All the Time
02	Trust Your Team
03	Review Everything, Test Everything
04	All Software Engineers are Created Equal
05	Doing the Project Right is Most Efficient
06	Part-1: Tools and Techniques
07	Part-2: Using Project Management Effectively
	Tools and Techniques
	Software Project Planning
08	Understand the Project Needs – Top15ics 8
20	Create the Project Plan – 12
24	Diagnosing Project Planning Problems – 04
	Estimation
27	Elements of a Successful Estimate – Topics 3
40	Wideband Delphin Estimation -13
43	Other Estimation Techniques – Topics 3
46	Diagnosing Scheduling Problems – Topics 3
	Project Schedules
58	Building the Project Schedule – Topics 12
60	Managing Multiple Projects – Topics 2
61	Use the Schedule to Manage Commitments
63	Diagnosing Scheduling Problems – Topics 2
	Review
75	Inspections – 12
78	Deskchecks – Topics 3
79	Walkthroughs
82	Code Reviews – Topics 3
84	Pair Programming – Topics 2
83	Use Inspections to Manage Commitments
86	Diagnosing review Problems – Topics 3
	Software Requirements
92	Requirements Elicitation – Topics 6
99	Use Cases – Topics 7
112	Software Requirements Specification – Topics 13
118	Change Control – Topics 6
121	Introduce Software requirements Carefully – Topics 3
123	Diagnosing Software Requirements Problems – Topics 2
	Design and Programming
125	Review the Design – Topics 2
140	Version Control with Subversion – Topics 15
143	Refactoring – Topics 3
150	Unit testing – Topics 7
152	Use Automation – Topics 2
153	Be Careful with Existing Projects
157	Diagnosing Design and Programming Problems – Topics 4

Software Project Management: Reference-2

	Essential Scrum: A Practical Guide to the Most Popular Agile Process
	Introduction
01	What is Scrum?
02	Scrum Origins
03	Why Scrum?
04	Genomica Results
10	Can Scrum Help You? – Topics 6
11	Closing
	Part-01: Core Concepts
	Scrum Framework
12	Overview
15	Scrum Roles – Topics 3
19	Scrum Activities and Artifacts – Topics 4
23	Scrum Activities and Artifacts – Topics 4
24	Closing
	Agile Principles
25	Overview
29	Variability and Uncertainty – Topics 4
34	Prediction and Adaptation – Topics 5
37	Validate Learning – Topics 3
41	Work in Process (WIP) – Topics 4
44	Progress – Topics 3
47	Performance – Topics 3
48	Closing
	Sprints
49	Overview
55	Timeboxed – Topics 6
61	Short Duration – Topics 6
63	Consistent Duration – Topics 2
69	No Goal-Altering Changes – Topics 6
73	Definition of Done – Topics 4
74	Closing
	Requirements and User Stories
75	Overview
76	Using Conversations
77	Progressive Refinement
80	What are User Stories? – Topics 3
81	Level of Detail
87	INVEST in Good Stories – Topics 6
88	Nonfunctional Requirements
89	Knowledge-Acquisition Stories
91	Gathering Stories – Topics 2
92	Closing
	Production Backlog
93	Overview
94	Product Backlog Items
98	Good Product Backlog Characteristics – Topics 4
101	Grooming – Topics 3

Software Metrics

Software Metrics: Reference-1

Software Metrics: Reference-2

Software Requirements

	Requirements Engineering for Software and Systems
	Introduction to Requirements Engineering
01	Motivation
02	What is Requirements Engineering?
03	You Probably Don't Do Enough Requirement Engineering
04	What are Requirements?
05	Requirements Engineering Activities
06	Bodies of Knowledge
07	The Requirements Engineer
08	Requirements Engineer Roles
09	Role of the Customer
10	Problems with Traditional Requirements Engineering
11	Difficulties in Enveloping System Behavior
12	Exercises
	References
	Preparing for Requirements Elicitation
13	Production Mission Statement
14	Encounter with a Customer
15	Identifying the System Boundaries
16	Stakeholders
17	Customer Wants and Needs
18	Why Do Customers Change Their Minds?
19	Stakeholder Prioritization
20	Communicating with Customers and Other Stakeholders
21	Stakeholder Negotiations
22	Uncovering Stakeholder Goals
23	Exercises
	References
	Requirements Elicitation
24	Introduction
25	Preparing for Requirements Elicitation
31	Elicitation Techniques Survey – Topics 6
37	Elicitation Techniques Survey – Topics 6
43	Elicitation Techniques Survey – Topics 6
47	Elicitation Techniques Survey – Topics 4
48	Eliciting Nonfunctional Requirements
49	Elicitation Summary
50	Eliciting Hazards
51	Exercise
	References
	Writing the Requirements Document
52	Requirements Agreement and Analysis
53	Requirements Representation
54	ISO/IEC/IEEE Standard 29148
55	UML/SysML
56	The Requirement Document – Topics ½
57	The Requirement Document – Topics ½
58	Behavioral Specifications

100	Anti-patterns in Requirements Management – Topics ½
111	Other Paradigms for Requirements Management
112	Standards for Requirements Management
113	Exercises
	References
	Value Engineering of Requirements
114	What, Why, When, and How of Value Engineering
115	Estimating Using COCOMO and Its Derivatives
116	Estimating Using Function Points
117	Requirements Feature Cost Justification – Topics ½
118	Requirements Feature Cost Justification – Topics ½
119	Putting It All Together
120	Exercises
	References
	Appendix
121	Software Requirements Specification for a Smart Home
122	Software Requirements for a Wastewater Pumping Station Web Well Control System
123	Unified Modeling Language (UML)
124	User Stories
125	Use Case
126	IBM DOORS Requirements Management Tool

Cloud Computing and Virtualization

93	Connectionless Packet Switching (Datagram Networks)
94	Router-Based Interconnectivity – Topics 3
95	Technical and Business Considerations – Topics 3
96	Data Center Technology
97	Virtualization
98	Standardization and Modularity
99	Automation
100	Remote Operation and Management
101	High Availability
102	Security-Aware Design, Operation, and Management
103	Facilities
104	Computing Hardware
105	Storage Hardware
106	Network Hardware – Topics 5
107	Other Considerations
108	Virtualization Technology
109	Hardware Independence
110	Server Consolidation
111	Resource Replication
112	Operating System-Based Virtualization
113	Hardware-Based Virtualization
114	Virtualization Management
115	Other Considerations
116	Web Technology
117	Basic Web Technology
118	Web Application
119	Multitenant Technology
120	Service Technology
121	Web Services
122	REST Services
123	Service Agents
124	Service Middleware | Case Study Example
	Fundamental Cloud Security
125	Basic Terms and Concepts
126	Confidentiality
127	Integrity
128	Authenticity
129	Availability
130	Threat
131	Vulnerability
132	Risk
133	Security Controls
134	Security Mechanisms
135	Security Policies
136	Threat Agents
137	Anonymous Attacker
138	Malicious Service Agent
139	Trusted Attacker
140	Malicious Insider
141	Cloud Security Threats
142	Traffic Eavesdropping

143	Malicious Intermediary
144	Denial of Service
145	Insufficient Authorization
146	Virtualization Attack
147	Overlapping Trust Boundaries
148	Additional Considerations
149	Flawed Implementations
150	Security Policy Disparity
151	Contracts
152	Risk Management | Case Study Example
	Part-02: Cloud Computing Mechanisms
	Cloud Infrastructure Mechanisms
153	Logical Network Perimeter | Case Study Example
154	Virtual Server | Case Study Example
155	Cloud Storage Device
156	Cloud Storage Levels
157	Network Storage Interfaces
158	Object Storage Interfaces
159	Database Storage Interfaces – Topics 2
160	Case Study Example
161	Cloud Usage Monitor
162	Monitoring Agent
163	Resource Agent
164	Polling Agent | Case Study Example
165	Resource Replication | Case Study Example
166	Ready-Made Environment | Case Study Example
	Specialized Cloud Mechanisms
167	Automated Scaling Listener | Case Study Example
168	Load Balancer | Case Study Example
169	SLA Monitor | Case Study Example – Topics 2
170	Pay-Per-Use Monitor | Case Study Example
171	Audit Monitor | Case Study Example
172	Failover System
173	Active-Active | Case Study Example
174	Hypervisor | Case Study Example
175	Resource Cluster | Case Study Example
176	Multi-Device Broker | Case Study Example
177	State Management Database | Case Study Example
	Cloud Management Mechanisms
178	Remote Administration System | Case Study Example
179	Resource Management System | Case Study Example
180	SLA Management System | Case Study Example
181	Billing Management System | Case Study Example
	Cloud Security Mechanisms
182	Encryption
183	Symmetric Encryption
184	Asymmetric Encryption | Case Study Example
185	Hashing | Case Study Example
186	Digital Signature | Case Study Example
187	Public Key Infrastructure (PKI) | Case Study Example
188	Identity and Access Management (IAM) | Case Study Example

189	Single Sign-On (SSO) | Case Study Example
190	Cloud-Based Security Groups | Case Study Example
191	Hardened Virtual Server Images | Case Study Example
	Part-03: Cloud Computing Architecture
	Fundamental Cloud Architectures
192	Workload Distribution Architecture
193	Resource Pooling Architecture
194	Dynamic Scalability Architecture
195	Elastic Resource Capacity Architecture
196	Service Load Balancing Architecture
197	Cloud Bursting Architecture
198	Elastic Disk Provisioning Architecture
199	Redundant Storage Architecture Fundamental Cloud Architectures
	Advanced Cloud Architectures
200	Hypervisor Clustering Architecture
201	Load Balanced Virtual Server Instances Architecture
202	Non-Disruptive Service Relocation Architecture
203	Zero-Disruptive Service Relocation Architecture
204	Cloud Balancing Architecture
205	Resource Reservation Architecture
206	Dynamic Failure Detection and Recovery Architecture
207	Bare-Metal Provisioning Architecture
208	Repaid Provisioning Architecture
209	Storage Workload Management Architecture
210	Case Study Example
211	Specialized Cloud Architectures
212	Direct I/O Access Architecture
213	Direct LUN Access Architecture
214	Dynamic Data Normalization Architecture
215	Elastic Network Capacity Architecture
216	Cross-Storage Device Vertical Tiering Architecture
217	Intra-Storage Device Vertical Tiering Architecture
218	Load Balanced Virtual Switches Architecture
219	Multipath Resource Access Architecture
220	Persistent Virtual Network Configuration Architecture
221	Redundant Physical Connection for Virtual Servers Architecture
	Part-04: Working with Clouds
	Cloud Delivery Model Considerations
222	Cloud Delivery Models: The Cloud Provider Perspective
223	Building IaaS Environments – Topics 4
224	Equipping PaaS Environments – Topics 3
225	Optimizing SaaS Environments | Security
	Cloud Delivery Models: The Cloud Consumer Perspective
226	Working with IaaS Environments
227	IT Resource Provisioning Considerations
228	Working with PaaS Environments
229	IT Resource Provisioning Considerations
230	Working with SaaS Services
231	Case Study Example
	Cost Metrics and Pricing Models
232	Business Cost Metrics

DevOps (Tools)

Learning DevOps can feel overwhelming, but with a structured plan, it becomes manageable. Below is a comprehensive, step-by-step **6-month guide** that covers everything from project management to monitoring, including all the tools we discussed.

This schedule is designed for a commitment of **15–20 hours per week**. Adjust the pace to fit your availability—it can be stretched to 8–9 months if you have less time.

📅 The 6-Month DevOps Learning Schedule

Phase	Month	Week	Topic	Key Tools & Technologies	Hands-On Project / Task
1. Foundations	**1**	1	**DevOps Culture & Project Management**	Agile, Scrum, Kanban, Jira, Trello, Confluence	Set up a Jira project board. Create user stories and a sprint backlog for a sample application.
		2	**Linux Fundamentals & Networking**	Linux CLI (Ubuntu/CentOS), Bash, SSH, Permissions, Systemd	Navigate the file system, manage users, set up a web server (Nginx/Apache) manually.
		3	**Version Control with Git**	Git, GitHub, GitLab, Branching (Git Flow)	Create a repo, commit changes, create branches, merge, and handle merge conflicts.
		4	**Scripting & Automation**	Bash Scripting, Python Basics	Write a Bash script to automate system backups. Write a Python script to parse log files.
Phase	**Month**	**Week**	**Topic**	**Key Tools & Technologies**	**Hands-On Project / Task**
2. CI/CD & Build	**2**	5	**Build Systems & Package Management**	**C++:** CMake, Conan **Python:** Poetry, pip **JavaScript:** Vite, npm	Set up a build system for a sample project in your chosen language. Manage dependencies.
		6	**Testing Fundamentals**	**C++:** Google Test **Python:** pytest **JavaScript:** Jest	Write unit tests and integration tests for your sample project. Aim for >80% code coverage.
		7	**Continuous Integration (CI)**	Jenkins, GitHub Actions, GitLab CI	Create a CI pipeline that automatically builds and tests your code on every push.
		8	**Continuous Delivery (CD)**	GitHub Actions, GitLab CI, ArgoCD, Spinnaker	Extend your pipeline to automatically deploy your application to a staging environment.
Phase	**Month**	**Week**	**Topic**	**Key Tools & Technologies**	**Hands-On Project / Task**

3. Containers & Orchestration	**3**	9	**Containerization with Docker**	Docker, Docker Compose, Container Registries	Containerize your sample application. Use Docker Compose to run multi-container apps.
		10	**Container Orchestration (Intro)**	Kubernetes (K8s), Minikube, kubectl	Deploy a containerized app to a local Kubernetes cluster. Learn Pods, Deployments, Services.
		11	**Advanced Kubernetes**	Helm, Ingress, Persistent Volumes, ConfigMaps	Package your app with Helm. Set up ingress for external access. Manage configuration.
		12	**Kubernetes in Production**	K3s, EKS, AKS, GKE	Deploy a scalable, highly available application on a managed Kubernetes service (e.g., EKS).
Phase	**Month**	**Week**	**Topic**	**Key Tools & Technologies**	**Hands-On Project / Task**
4. Infrastructure as Code	**4**	13	**Configuration Management**	Ansible, Chef, Puppet	Use Ansible to automate the setup of a web server (install Nginx, configure firewall).
		14	**Infrastructure as Code (IaC)**	Terraform, Pulumi, AWS CloudFormation	Provision a cloud environment (VPC, subnets, EC2 instances) using Terraform.
		15	**Cloud Platforms (AWS/Azure/GCP)**	AWS (EC2, S3, RDS, VPC), Azure, GCP	Deploy a sample application to the cloud. Use managed services like RDS for a database.
		16	**GitOps**	ArgoCD, Flux	Implement GitOps: use Git as the single source of truth for your infrastructure and application state.
Phase	**Month**	**Week**	**Topic**	**Key Tools & Technologies**	**Hands-On Project / Task**
5. Monitoring & Debugging	**5**	17	**Logging**	ELK Stack (Elasticsearch, Logstash, Kibana), Loki	Centralize logs from your application and infrastructure. Create dashboards in Kibana.
		18	**Metrics & Monitoring**	Prometheus, Grafana	Set up Prometheus to collect metrics (CPU, memory, request latency). Visualize them in Grafana.
		19	**Alerting & Incident Management**	Alertmanager, PagerDuty, Opsgenie	Configure alerts for high error rates or system downtime. Integrate with PagerDuty for on-call rotations.
		20	**Debugging & Distributed Tracing**	Jaeger, Zipkin, Sentry, GDB/Valgrind (C++)	Implement distributed tracing to track requests

					across microservices. Use Sentry for error tracking.
Phase	**Month**	**Week**	**Topic**	**Key Tools & Technologies**	**Hands-On Project / Task**
6. Advanced & Integration	**6**	21	**Security (DevSecOps)**	SonarQube, Snyk, Trivy, OWASP ZAP	Integrate security scanning into your CI/CD pipeline (SAST, DAST, dependency scanning).
		22	**AI & MLOps (Optional)**	MLflow, Kubeflow, NVIDIA Triton, DVC	Learn how DevOps principles apply to machine learning. Set up a model training and serving pipeline.
		23	**Final Project (Part 1)**	**All tools combined**	**Project:** Design and implement a complete end-to-end CI/CD pipeline for a microservices application on Kubernetes with monitoring and alerting.
		24	**Final Project (Part 2)**	**All tools combined**	**Project:** Polish the project, document everything, and present it as a portfolio piece.

🗺️ The Recommended Learning Path

This schedule follows a proven, logical progression:

1. **Start with the Fundamentals**: Linux, networking, and a scripting language like **Bash** or **Python** are your foundation. Without these, the rest will be very difficult.
2. **Master Version Control**: Get comfortable with **Git** and a platform like **GitHub** or **GitLab**. This is non-negotiable.
3. **Learn Build & Test**: Understand how to compile code and run tests automatically. This prepares you for CI/CD.
4. **Build CI/CD Pipelines**: Use a tool like **GitHub Actions** or **Jenkins** to automate the build, test, and deployment process.
5. **Learn Containerization**: Begin with **Docker** to understand how to package applications.
6. **Move to Orchestration**: Once comfortable with Docker, learn **Kubernetes** to manage containers at scale.
7. **Tackle Infrastructure as Code**: Learn **Terraform** to provision the infrastructure your containers will run on.
8. **Implement Monitoring & Logging**: Finally, learn **Prometheus** and **Grafana** to observe your running applications.

💡 Tips for Success

- **Hands-on > Theory**: DevOps is a practical field. Spend at least 70% of your time doing hands-on labs and projects.
- **One Tool at a Time**: Don't try to learn everything at once. Master one tool before moving to the next.
- **Use Free Resources**:
  - **Roadmap.sh**: The DevOps Roadmap is an excellent visual guide.
  - **GitHub**: Explore the many free DevOps learning paths and resources available.
  - **Cloud Free Tiers**: AWS, Azure, and GCP all offer free tiers perfect for learning.
- **Build a Portfolio**: The final project is crucial. It demonstrates your ability to apply all the tools in a real-world scenario.
- **Join a Community**: Engage with DevOps communities on Reddit (r/devops), LinkedIn, or Discord. Ask questions and learn from others.

🤔 What About My Programming Language?

You can apply this roadmap with **any** language (C++, Python, JavaScript, etc.). The tools for **build, test, and package management** will change, but the CI/CD, containerization, and cloud concepts remain the same. Choose one language to focus on for your projects.

This schedule provides a clear, step-by-step path. Would you like me to elaborate on any specific week or tool?

Below is a comprehensive, end-to-end DevOps tools table, classified from **Project Management** to **Monitoring**. For each category, I've highlighted the **most popular** tools specifically for **C++**, **Python**, **Web (JavaScript)**, and **AI/ML** development in 2025.

🧭 The Complete DevOps Toolchain (Classified by Stage)

Stage	Category	C++	Python	JS/TypeScript	AI / ML
Plan	**Project Management**	Jira, Linear, Azure DevOps	Jira, Linear, Azure DevOps	Jira, Linear, Azure DevOps	Jira, Linear, Azure DevOps
Plan	**Documentation & Knowledge**	Confluence, Notion, GitHub Wiki	Confluence, Notion, GitHub Wiki	Confluence, Notion, GitHub Wiki	Confluence, Notion, GitHub Wiki
Code	**Version Control (SCM)**	Git, GitHub, GitLab, Bitbucket	Git, GitHub, GitLab, Bitbucket	Git, GitHub, GitLab, Bitbucket	Git, GitHub, GitLab, Bitbucket
Code	**IDE & Local Environment**	Visual Studio, CLion, VS Code	PyCharm, VS Code, Jupyter	VS Code, WebStorm, Cursor	VS Code, Jupyter, Cursor
Code	**AI Coding Assistant**	GitHub Copilot, SourceCraft	GitHub Copilot, Amazon Q	GitHub Copilot, Cursor AI	GitHub Copilot, Cursor AI, Sourcegraph Cody
Build	**Package Manager**	Conan, vcpkg, Build2	pip, Poetry, Conda	npm, yarn, pnpm	pip, Conda, Poetry
Build	**Build System / Compiler**	CMake, Ninja, Bazel, Make	Setuptools, Poetry, PyInstaller	Vite, Webpack, esbuild, Rollup	Setuptools, Poetry, Bazel
Build	**Code Quality & Linting**	Clang-Tidy, Clang-Format, SonarQube	Ruff, Flake8, Black, mypy	ESLint, Prettier, SonarQube	Ruff, Pylint, mypy
Test	**Unit / Integration Testing**	Google Test, Catch2, CTest	pytest, unittest, nose	Jest, Vitest, Mocha	pytest, unittest
Test	**E2E / Browser Automation**	Selenium, Playwright	Selenium, Playwright	Playwright, Cypress, Selenium	Selenium, Playwright
Test	**Performance / Load Testing**	JMeter, Gatling, Valgrind	JMeter, Locust	JMeter, k6	JMeter, k6
Test	**Security / SAST**	SonarQube, CodeQL	Bandit, SonarQube, CodeQL	SonarQube, CodeQL, Snyk	Bandit, SonarQube, CodeQL
CI	**Continuous Integration**	GitHub Actions, GitLab CI, Jenkins	GitHub Actions, GitLab CI, Jenkins	GitHub Actions, GitLab CI, CircleCI	GitHub Actions, GitLab CI, Jenkins
CD	**Continuous Delivery / GitOps**	ArgoCD, Flux	ArgoCD, Flux, Spinnaker	ArgoCD, Flux, Vercel, Netlify	ArgoCD, Flux, Kubeflow
Deploy	**Containerization**	Docker, Podman	Docker, Podman	Docker, Podman	Docker, Podman, NVIDIA Container Toolkit
Deploy	**Container Orchestration**	Kubernetes, K3s	Kubernetes, K3s	Kubernetes, K3s	Kubernetes, K3s

Deploy	**Infrastructure as Code (IaC)**	Terraform, Pulumi	Terraform, Pulumi, AWS CDK	Terraform, Pulumi	Terraform, Pulumi
Deploy	**Configuration Management**	Ansible, Chef	Ansible, SaltStack, Fabric	Ansible	Ansible
Deploy	**ML Model Serving**	—	—	—	**NVIDIA Triton, TensorFlow Serving, BentoML**
Deploy	**ML Pipeline Orchestration**	—	—	—	**Apache Airflow, Kubeflow, MLflow, Metaflow**
Operate	**Debugging & Tracing**	GDB, Valgrind, AddressSanitizer	pdb, py-spy, Sentry	Chrome DevTools, Sentry	LangSmith, Weights & Biases
Operate	**Distributed Tracing**	Jaeger, Zipkin	Jaeger, Zipkin	Jaeger, Zipkin	Jaeger, Zipkin
Monitor	**Metrics & Monitoring**	Prometheus + Grafana	Prometheus + Grafana	Prometheus + Grafana	Prometheus + Grafana
Monitor	**Logging**	ELK Stack (Elasticsearch, Logstash, Kibana)	ELK Stack	ELK Stack	ELK Stack
Monitor	**Observability Platforms**	Datadog, New Relic, Dynatrace	Datadog, New Relic, Dynatrace	Datadog, New Relic, Dynatrace	Datadog, New Relic, Dynatrace
Monitor	**Incident Alerting**	Alertmanager, PagerDuty, Opsgenie	Alertmanager, PagerDuty, Opsgenie	Alertmanager, PagerDuty, Opsgenie	Alertmanager, PagerDuty, Opsgenie

🧠 AI's Role IN DevOps (Not Just FOR AI)

AI is now embedded **inside** DevOps tools themselves:

Tool	AI Feature
GitHub Copilot	Writes YAML pipelines, Dockerfiles, and Terraform code.
Amazon Q Developer	Expert in IaC (Terraform/CDK) and AWS.
Dynatrace / Datadog	AI-powered anomaly detection and root cause analysis.
Snyk AI	AI-assisted security vulnerability fixing.

🔑 Key Observations by Language

Language	Build System	Testing	CI/CD	Special Notes
C++	**CMake + Conan/vcpkg** is the gold standard. Bazel for massive monorepos.	**Google Test** for unit tests. **Valgrind & AddressSanitizer** for memory debugging.	**GitHub Actions** & **GitLab CI** are most popular.	C++ DevOps is heavily focused on **build speed** (Ninja, ccache) and **dependency management**.
Python	**Poetry** is modern standard. **Setuptools** for legacy.	**pytest** is the undisputed king. **Ruff** is the new ultra-fast linter.	**GitHub Actions** is #1. **Ansible** & **Fabric** for automation.	Python shines in **automation scripts** and **cloud SDKs** (boto3, kubernetes client).
JavaScript	**Vite** is now #1 (98% usage). **esbuild** (88%) and **Rollup** (79%) are also huge. Webpack is declining (55%).	**Jest** is most popular. **Vitest** is gaining fast.	**GitHub Actions** & **GitLab CI** dominate. **Vercel/Netlify** for frontend hosting.	JavaScript DevOps is about **fast builds** and **frontend deployment**. **TypeScript** is now assumed.
AI / ML	**Conda** & **Poetry** for Python dependencies.	**pytest** for code. **Great Expectations** for data validation.	**GitHub Actions** for code CI. **Kubeflow**, **Airflow**, **MLflow** for ML pipelines.	**MLOps** is a superset of DevOps. Key additions: **Model Serving** (Triton, TF Serving), **Experiment Tracking** (MLflow, Weights & Biases), and **Data Versioning** (DVC, LakeFS).

📌 Summary

Your Original List	DevOps Category	Tools (2025)
Git	**Version Control**	Git, GitHub, GitLab
C++, Python, JavaScript	**Build + Test + Code Quality**	CMake/Conan (C++), Poetry/pytest (Python), Vite/Jest (JS)
Web-based environment, IDE	**Local Dev Environment**	VS Code, Cursor, WebStorm, PyCharm
Library, packages setup	**Package Managers**	Conan/vcpkg, pip/Poetry, npm/yarn
Build system	**Build Tools**	CMake/Ninja, Vite/esbuild, Setuptools
Integration	**CI/CD**	GitHub Actions, GitLab CI, Jenkins
Cloud computing	**IaC + Cloud**	Terraform, Kubernetes, AWS/Azure/GCP
Testing	**Testing Frameworks**	Google Test, pytest, Jest
Debugging	**Debugging + Tracing**	GDB/Valgrind, pdb/Sentry, Jaeger/Zipkin
Design Management	**Architecture (Not DevOps)**	Handled by Software Architects
Consumer Satisfaction	**Product Management (Not DevOps)**	Handled by PMs & UX Researchers
Project Management	**Planning & Tracking**	Jira, Linear, Azure DevOps
Monitoring	**Observability**	Prometheus + Grafana, ELK, Datadog

This table gives you a **complete, language-specific roadmap** for mastering DevOps in 2025. Would you like me to go deeper into any specific category (e.g., how to set up a CI/CD pipeline for C++ with CMake + GitHub Actions)?

Prompt Engineering and Workspace Automation

You've hit on two of the most practical and in-demand applications of AI today: **prompt engineering** and **workplace automation**. They are deeply connected. Prompt engineering is the skill that makes AI automation possible—it's the "how" you instruct an AI to perform a task for you.

Let's break down what this looks like in a modern workplace and how you can build these skills.

🤖 What is "Workplace Automation" with AI?

Workplace automation means using AI to handle repetitive, time-consuming tasks. This isn't about replacing people, but about augmenting their capabilities to focus on more strategic work. It's about turning a manual process into a flow diagram and inserting "small automations that deliver quick ROI".

In practice, this could mean using AI to:

- **Draft and summarize emails** and instant messages.
- **Generate reports** from raw data.
- **Create content** for marketing, social media, or internal communication.
- **Automate business processes** like lead generation or customer support.
- **Assist in DevOps** by generating infrastructure code, scripts, and CI/CD pipeline configurations.

🧠 The Role of Prompt Engineering

Prompt engineering is the art and science of crafting effective instructions for AI models like ChatGPT, Gemini, or Claude. A well-crafted prompt is the difference between a generic, unhelpful response and a precise, actionable output.

The skill is becoming so fundamental that some call it "the new language of the workplace". It's no longer just a command; it's a core competency for navigating the future of work.

📚 A Roadmap to Mastering Prompt Engineering & Automation

Here is a structured, 4-step roadmap to go from a beginner to a professional who can build AI-powered automations.

Phase 1: The Foundation (1-2 Months)

Goal: Understand the core principles and start getting consistent results.

- **Step 1: Master the Fundamentals**
    - **Be Explicit and Clear**: Modern AI models respond exceptionally well to clear, explicit instructions. Vague prompts create vague results.
    - **Provide Context**: The more the model knows about the situation, the better it performs.
    - **Define the Output Format**: Tell the AI exactly how you want the information returned (e.g., "as a bulleted list," "in JSON format," or "as a table").
    - **Use Key Techniques**: Learn and practice these fundamental prompting methods:
        - **Zero-shot Prompting**: Asking the AI to perform a task without any examples.
        - **Few-shot Prompting**: Providing a few examples of the desired input and output before asking the AI to perform the task.

        - **Role-based Prompting**: Assigning a role to the AI (e.g., "Act as a senior DevOps engineer...").
        - **Chain-of-Thought (CoT) Prompting**: Asking the AI to "think step-by-step" to solve a complex problem.

- **Step 2: Practice with a Simple Tool**
    - Start with a free, user-friendly platform like **Google AI Studio** or **ChatGPT** to experiment with different prompts and techniques.

Phase 2: Building Practical Automations (2-3 Months)

Goal: Learn to connect AI to other tools and build simple, automated workflows.

- **Step 3: Chain Prompts into Workflows**
    - Learn to use automation platforms like **Zapier** or **Make** (formerly Integromat). These tools allow you to connect AI to hundreds of other apps and services, creating powerful, multi-step automations.
    - **Example**: Create a workflow where an email attachment is automatically sent to an AI, which summarizes it and then posts the summary to a Slack channel.
- **Step 4: Integrate LLM APIs**
    - Learn how to interact with AI models programmatically using their APIs (e.g., OpenAI API, Anthropic API). This is where the real power of automation lies.
    - **Example**: Write a simple Python script that uses the OpenAI API to automatically generate a daily report from a database query.

Phase 3: Advanced Prompt Engineering (2-3 Months)

Goal: Move beyond simple prompts to create robust, reliable, and reusable AI systems.

- **Step 5: Treat Prompts Like Code**
    - This is a crucial shift. Prompts should be **version-controlled, reviewed, tested, and observable**, just like application code.
    - Learn to use **YAML** or **JSON** templates to structure and standardize your prompts.
    - Use **Pydantic** to validate and enforce the structure and quality of the AI's output, ensuring it's always in a predictable format.
- **Step 6: Explore Automation Frameworks**
    - Get hands-on with specialized toolkits:
        - **Prompt Engineer Toolkit**: A production-grade framework for creating and managing prompts, with support for CI/CD and Docker.
        - **PromptForge**: A TypeScript SDK for designing, optimizing, versioning, and evaluating prompts.

Phase 4: Enterprise & DevOps Integration (Ongoing)

Goal: Apply your skills at scale within a business or technical environment like DevOps.

- **Step 7: Apply to DevOps**
    - This is where your DevOps knowledge becomes a superpower. Use AI to:

        - **Generate Infrastructure as Code**: Write prompts to generate Terraform, CloudFormation, or Bicep templates.
        - **Create Scripts**: Generate and translate complex Azure CLI, PowerShell, or Bash scripts from natural language descriptions.
        - **Build CI/CD Pipelines**: Use AI to help you draft and refine pipeline configurations (e.g., GitHub Actions, GitLab CI).
    - **Example**: A strong prompt for a DevOps task is specific. Instead of "write a script," use "Write a PowerShell script to get all stopped EC2 instances in the 'production' AWS account and send a notification to an SNS topic named 'DevOps-Alerts'."
- **Step 8: Build Organization-Wide Solutions**
    - Learn to build **multi-agent prompt engineering systems** that standardize and automate dynamic prompt execution across entire teams.
    - Use platforms like **Azure AI Foundry** and **Prompt Flow** to govern and standardize prompts for enterprise chatbots and applications.

💡 **Key Takeaway**

The future of prompt engineering isn't just about being a specialist who writes prompts. It's about being a **strategic professional** who can identify automation opportunities and build reliable, scalable AI-powered systems.

Your path from DevOps to AI is a powerful one. Your understanding of automation, infrastructure, and code will make you exceptionally effective at building the next generation of AI-augmented workplaces.

Would you like me to dive deeper into any of these phases, such as the specific Python code for an automation or a detailed example of a DevOps-focused prompt?

Calculus and Analytical Geometry

Calculus and Analytical Geometry: Reference-1

Calculus and Analytical Geometry: Reference-2